Design and Evaluation of SafeGuardian: A Service-Oriented Emergency SOS and Asynchronous Email Notification System

Dr. C. Srikanth
Department of Computer Science and Engineering
KLH University, Hyderabad, India
Email: csrikanth@klh.edu.in

K. Vamsidhar
Department of Computer Science and Engineering
KLH University, Hyderabad, India
Email: 2420030341@klh.edu.in

***Abstract*—Women-safety applications provide emergency alerting and location sharing, but an alert workflow must also preserve incident information and coordinate communication with registered contacts. Existing approaches explore mobile applications, Internet of Things devices, GPS tracking, and different emergency-trigger mechanisms; however, these approaches vary in platform, communication channel, and implementation. This paper presents SafeGuardian, a web-based emergency SOS system designed to separate incident creation from email notification delivery. The system integrates session-based authentication, emergency-contact management, browser geolocation, persistent incident records, and an independent notification worker. The frontend is built using HTML, CSS, and JavaScript. The backend runs on Node.js and Express, with PostgreSQL used to store application data. When an authenticated user activates SOS, the application records the incident and creates a notification record for each configured emergency contact. A separate worker processes pending records through email and updates their recorded status. The prototype was evaluated through functional checks covering authentication, location display, contact management, SOS activation, email notification, incident resolution, and history retrieval. The available validation record reports successful completion of the tested core workflow and no failed notification records in the final controlled validation. These results demonstrate functional feasibility in the tested environment but do not establish production-scale reliability, statistical reliability, or performance under load. The paper presents the system architecture, workflow, evaluation measures, and limitations.**

***Index Terms*—Women's safety, emergency response, SOS application, GPS tracking, asynchronous notification, web application.**

I. Introduction

Women's safety is an important application area for software systems that provide emergency communication, location sharing, and contact-based assistance. A safety application should enable a user to initiate an alert and communicate useful information to trusted contacts. The application must also handle the subsequent operations, including validating the user, recording the incident, retrieving the configured contacts, sending the notification, and preserving the outcome for later inspection.

Existing women's-safety solutions use different technologies to support these operations. Mobile applications offer convenient interfaces for users to access emergency features and share their location. IoT-based devices can also use physical buttons and sensors to initiate emergency alerts. The capabilities of these systems depend on the hardware and software they use, the availability of network connectivity, and the communication channels through which alerts are delivered.

A further software-engineering consideration is what happens after an SOS request reaches the application server. Sending an email requires communication with an external mail service. If this operation is performed synchronously within the emergency request, the request may remain dependent on the response time of that service. A separate notification-processing component offers an alternative: the application can persist the emergency incident and its notification tasks, after which a worker processes the tasks independently.

SafeGuardian explores this approach through a web-based women's-safety prototype. The user explicitly activates SOS through an authenticated interface. The application captures browser location when permission and device support are available, stores the incident in PostgreSQL, and creates a notification record for each configured emergency contact. An independent worker processes pending email notifications and records their outcomes.

The research problem is therefore not to claim that email is universally superior to SMS or mobile notifications. Instead, it is to design and validate a traceable emergency workflow in which incident creation and external email delivery are separate operations.

The main contributions are as follows:

- A web-based emergency SOS architecture integrating authentication, contact management, browser geolocation, and persistent incident records.
- A database-backed notification workflow that separates SOS processing from email delivery through an independent worker.
- A mathematical representation of incidents, notification records, and functional evaluation measures.
- A functional evaluation of the implemented prototype, accompanied by an explicit discussion of evidence limi-

tations and future testing requirements.

The system does not implement machine-learning-based threat detection, automatic distress recognition, or a guaranteed emergency-service connection. Its focus is the software workflow initiated by an authenticated user.

The remainder of this paper reviews related women's-safety systems, defines the problem and objectives, explains the proposed methodology and architecture, describes the implementation and evaluation approach, discusses the observed results, and concludes with limitations and future work.

II. Related Work

Women's-safety research includes mobile applications, sensor-based devices, location-aware alerting, and integrated emergency-response platforms. These approaches share the goal of helping a user communicate an emergency, but they differ in how an alert is triggered, which information is transmitted, and how the receiving party is contacted.

A. Mobile Applications and Emergency Alerts

Saravanan and Marjudi describe a Women SOS Safety App that supports emergency communication and GPS information for predefined contacts [2]. This work is relevant to SafeGuardian because both systems use an application-based workflow to notify trusted contacts. SafeGuardian focuses more specifically on persistent incident records and contact-level notification records processed by an independent worker.

An integrated mobile application approach proposes separate applications for women and police, together with computer-aided dispatch capabilities and location-aware emergency information [9]. This direction broadens the workflow from contacting trusted individuals to coordinating a response involving public-safety personnel. SafeGuardian currently has a narrower scope and does not claim integration with police dispatch infrastructure.

Recent mobile safety work also investigates live location, video, and audio streaming as part of emergency response [4]. Such features can provide additional context during an incident, but they also introduce additional data, privacy, connectivity, and implementation requirements. SafeGuardian concentrates on location-aware SOS incidents and email notification rather than media streaming.

B. IoT, GPS, and Hardware-Based Systems

IoT-based women's-safety research explores devices that use sensors and communication modules to identify or communicate possible emergencies. Farooq *et al.* present a systematic literature review of IoT-based women's-safety devices, examining their features, sensors, communication technologies, and limitations [8]. Their review identifies a broad range of design choices and discusses the need for systems that reduce interaction requirements and improve practical usability.

The Women Safety System using IoT explores connected technology for women's-safety applications [3]. Related device-oriented research combines GPS and GSM modules with sensing hardware to support location transmission and emergency communication [10]. Another SOS device approach also focuses on automatic emergency alerts and geolocation sharing through GSM/GPS integration [5].

These systems illustrate the value of hardware-based triggers and cellular communication, particularly where a smartphone interface may be difficult to use. SafeGuardian differs in that it is a browser-based software prototype. It relies on explicit user activation and browser geolocation instead of dedicated sensing hardware or automatic threat recognition.

C. Recent Integrated and Cross-Platform Approaches

Recent studies have examined ways to combine mobile applications, location tracking, and connected devices to support women's safety. A cross-platform women's-safety application describes real-time alerting and GPS tracking [6]. The Thozha system explores a mobile and IoT-based safety ecosystem [7]. These approaches demonstrate the diversity of possible system architectures, from software applications to integrated hardware and mobile environments.

The Rakshak framework discusses a multi-layered approach to women's safety with alternative emergency-trigger mechanisms [1]. Such mechanisms address the challenge of initiating an alert in circumstances where conventional manual interaction may be difficult. SafeGuardian does not attempt to reproduce these triggering mechanisms; instead, it investigates the handling of an SOS event after a user activates it.

D. Research Gap and Positioning

The reviewed studies demonstrate several important directions: mobile emergency applications, GPS-based location sharing, IoT sensing, hardware-triggered alerts, and integrated response systems. Their features should not be assumed to be directly comparable because the platforms, communication channels, and evaluation methods differ.

SafeGuardian addresses a narrower implementation question: how can a web application preserve an SOS incident and create independently traceable notification tasks without making the incident workflow entirely dependent on immediate email delivery? Its design uses persistent notification records and a separate worker to process them.

This is a system-design and functional-evaluation contribution, not a claim that existing women's-safety applications lack asynchronous processing. The current evaluation also does not establish that SafeGuardian is more reliable or faster than the cited systems. A controlled comparative study would be needed to support such a conclusion.

III. Problem Statement and Objectives

The problem addressed in this work is the design of a lightweight emergency SOS application that records an incident, associates it with registered emergency contacts, and processes email notifications independently of the initial SOS request.

The system has the following objectives:

1) Authenticate users before allowing access to protected emergency functions.

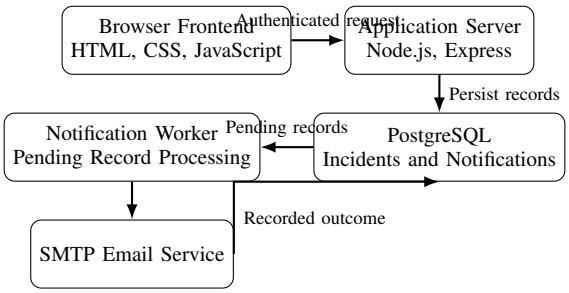


Fig. 1. High-level architecture of SafeGuardian.

2) Acquire browser location when the user grants permission and the device supports geolocation.
3) Allow users to manage their emergency contacts.
4) Persist SOS incidents and maintain their active or resolved state.
5) Create an individual notification record for each configured emergency contact.
6) Process pending email notifications using a separate worker.
7) Retain notification status and incident history for inspection.

IV. Proposed Methodology

A. Overall Architecture

SafeGuardian consists of four primary components: a browser frontend, an application server, a PostgreSQL database, and an independent notification worker. An SMTP service is used by the worker to send email notifications.

The browser frontend provides the user interface for authentication, contact management, SOS activation, and incident history. When the user activates SOS, the frontend obtains the location when browser permission and device support are available and sends the request to the application server.

The application server validates the authenticated session and request data. It creates an incident record and inserts notification records for the user's configured emergency contacts. The database preserves these records so that notification processing can take place independently of the original HTTP request.

The notification worker periodically checks for pending records. It attempts to send the corresponding emails and updates each notification record based on the observed result. The recorded outcome makes the notification workflow inspectable after the SOS request has completed.

B. Incident and Notification Model

Let U represent the set of users, C_u the set of emergency contacts registered by user u, and I the set of incidents. An incident is represented as

$$i = (u, \phi, \lambda, t_c, s, t_r), \tag{1}$$

Algorithm 1: SOS Creation and Notification Processing
Input: Authenticated user u, available coordinates (ϕ, λ).
Output: Incident record or validation response.
1. Validate the user session and request data.
2. Check whether the user has an active incident.
3. If a conflicting active incident exists, return the appropriate response.
4. Create the incident record.
5. Retrieve the user's configured emergency contacts C_u.
6. Create a pending notification record for each configured contact.
7. Persist the incident and notification records.
8. Return the incident result to the frontend.
9. Independently, the worker retrieves pending notifications.
10. Attempt email delivery for each eligible notification.
11. Update the notification state and applicable timestamps.
12. Continue processing according to the worker implementation.

where u is the user identifier, ϕ is latitude, λ is longitude, t_c is the incident creation time, s is the incident state, and t_r is the resolution time when applicable.

The incident state belongs to the set

$$s \in \{\text{ACTIVE}, \text{RESOLVED}\}.$$

For an incident i belonging to user u, the expected notification set is

$$N_i = \{n_{i,c} : c \in C_u\}, \qquad |N_i| = |C_u|. \tag{2}$$

Each notification is associated with an incident and an emergency contact. Its state is represented by

$$q \in \{\text{PENDING}, \text{SENT}, \text{FAILED}\}.$$

A notification is initially created as pending. The worker attempts delivery and records the outcome. The existence of a status field does not by itself guarantee exactly-once delivery or successful retry after every failure. Those properties depend on the actual processing and database logic.

C. SOS Workflow

The SOS workflow begins when an authenticated user activates the emergency control. The application obtains the location when available and submits the incident request to the server. After receiving the request, the server validates the submitted information and checks whether the user already has an active incident. If the request is valid, it saves a new incident and creates notification records for the user's configured emergency contacts.

The notification worker operates independently. It retrieves pending notification records, attempts email delivery, and updates their recorded states. This separation allows the system to preserve the emergency event and its notification tasks without requiring all external email operations to finish before the initial workflow returns.

D. SOS Processing Algorithm

The algorithm describes the intended logical workflow. Concurrency protection, transaction boundaries, retry behavior, and duplicate-processing prevention must be verified against the implementation before stronger guarantees are claimed.

E. Functional Evaluation Measures

For m tested SOS attempts, the observed SOS activation success ratio is

$$S_{\mathrm{SOS}} = \frac{n_{\mathrm{SOS,success}}}{m}, \tag{3}$$

where $n_{\mathrm{SOS,success}}$ is the number of attempts that produced the expected incident.

If e notification records were expected and g were created, notification-generation success is

$$S_{\mathrm{queue}} = \frac{g}{e}, \qquad e > 0. \tag{4}$$

For a email attempts and d successfully sent messages, the observed email-send success ratio is

$$S_{\mathrm{email}} = \frac{d}{a}, \qquad a > 0. \tag{5}$$

When every attempt is classified as sent or failed, the corresponding failure ratio is

$$F_{\mathrm{email}} = 1 - S_{\mathrm{email}}.$$

These ratios describe the observed test results. They should be reported together with their denominators, trial conditions, and observation period before they are interpreted as reliability measures.

When timestamps are available, the queue-to-send interval for notification j can be defined as

$$L_j = t_{\mathrm{sent},j} - t_{\mathrm{created},j}.$$

This measures the interval between the application's recorded timestamps. It does not measure when the recipient reads the email. No latency value is claimed in this paper because a complete measured latency dataset was not available in the supplied validation record.

F. Design Choices

The web-based interface avoids requiring users to install a dedicated mobile application for the prototype. Browser geolocation provides coordinates when supported and permitted by the user. PostgreSQL stores incident and notification records so that the system can preserve state across separate processing steps.

The independent worker separates email delivery from the main SOS request. This design provides a place to track notification processing, but it introduces additional concerns such as worker availability, concurrent processing, SMTP timeouts, and retry handling. These issues require dedicated failure and load testing.

The application uses explicit user activation rather than automatic threat recognition. It does not claim to infer danger from sensor data, detect distress using machine learning, or automatically contact emergency services.

V. Implementation and Security

The frontend uses HTML, CSS, and JavaScript. The application server uses Node.js and Express, while PostgreSQL stores user-related information, emergency contacts, incidents, and notification records. Session-based authentication protects user-specific operations, and bcrypt is used for password hashing. The independent notification worker sends emails through an SMTP service.

The implementation supports login, contact management, SOS activation, location display, incident resolution, and history retrieval. The final notification workflow uses email; SMS and WhatsApp are not part of the described implementation.

The browser location workflow depends on permission, browser support, and device availability. Email delivery depends on the SMTP provider and network connectivity. Sensitive configuration, including database credentials and mail-service credentials, should be managed outside source code.

The prototype is intended for academic evaluation. It should not be treated as a replacement for official emergency services or as a guarantee that every notification will be delivered under all circumstances.

VI. Experimental Setup and Evaluation Method

The available project validation record describes functional checks of authentication, location display, emergency-contact management, SOS activation, email notification, incorrect-password rejection, incident resolution, and incident history. The project team reported that all nine website functional checks passed, including the active-incident workflow checks.

The evaluation focuses on whether the implemented workflow behaves as expected in the tested environment. It covers the creation of an SOS incident, the acquisition and display of location when available, the creation of notification records, email processing, and the availability of incident history. Authentication checks also examine rejection of an incorrect password and the incident-resolution workflow.

The available records do not contain a complete per-trial dataset, exact repetition counts for every metric, a detailed hardware specification, or measured latency distributions. Accordingly, the results are reported as functional observations rather than statistically representative reliability estimates. No load-test results or comparison against a measured synchronous baseline are claimed.

For a reproducible follow-up evaluation, each trial should record the operating system, runtime and database versions, number of configured contacts, location-permission state, incident creation time, notification creation time, notification outcome, and applicable timestamps. Repeated tests should also include controlled failures, such as SMTP timeouts and worker restarts, before any reliability or resilience claim is made.

VII. Results and Discussion

The supplied validation record reports normalized observed values of 1.00 for SOS activation success, location acquisition success, notification creation success, and email delivery

TABLE I
OBSERVED RESULTS REPORTED IN THE VALIDATION RECORD

Metric	Reported value
SOS activation success	1.00
Location acquisition success	1.00
Notification creation success	1.00
Email delivery success	1.00
Notification failure rate	0.00

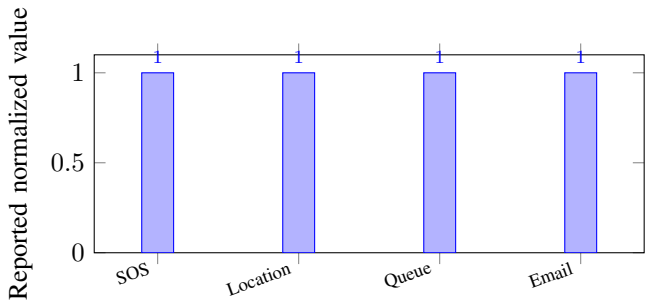


Fig. 2. Normalized success values reported in the available validation record. Trial counts were not supplied.

success during the final controlled validation. It also reports a notification failure rate of 0.00 for that validation. The original record does not provide the underlying trial counts for each metric; therefore, these values are reported observations rather than production reliability estimates.

The reported observations indicate that the intended core workflow completed successfully during the documented validation. The architecture separates incident creation from external email processing, while persistent notification records allow individual contact outcomes to be inspected. These characteristics support the functional feasibility of the implemented design in the tested environment.

The literature provides context for the range of women's-safety system designs. Mobile applications emphasize accessible interfaces and location-based alerts, while IoT and hardware-based systems explore sensing and alternative trigger mechanisms [8]–[10]. Recent systems also investigate richer emergency information and combinations of mobile and connected-device capabilities [4], [7]. SafeGuardian's narrower design is intended to complement these directions by concentrating on persistent incident management and asynchronous email notification.

These approaches cannot be quantitatively ranked using the evidence currently available. Their evaluation conditions, datasets, platforms, and reported metrics differ. The current SafeGuardian results also lack complete trial denominators and latency measurements. It would therefore be inappropriate to claim that SafeGuardian is faster, more accurate, or more reliable than the cited systems.

Further evaluation should examine concurrent SOS requests, multiple configured contacts, invalid recipient addresses, SMTP outages, worker restarts, and mixed delivery outcomes. Measuring request latency and queue-to-send intervals under controlled workloads would help determine the operational effect of asynchronous processing. Until those measurements are collected, the results should be interpreted as functional validation rather than a performance benchmark.

VIII. ERROR ANALYSIS AND LIMITATIONS

The current evidence has several limitations. First, the available validation record does not include raw per-trial observations or the complete denominators for the reported success ratios. The normalized values therefore cannot be interpreted as statistically representative estimates of reliability.

Second, no measured latency distribution, load-test result, or synchronous baseline is available. The use of a separate worker does not, by itself, demonstrate a quantitative performance improvement.

Third, email delivery depends on the SMTP provider and network conditions. A successful send operation does not prove that the recipient received, opened, or read the message. Browser geolocation also depends on user permission and device availability.

Fourth, concurrency behavior, retry policy, and duplicate processing require testing against the actual worker and database implementation. A status field and attempt counter alone do not establish exactly-once delivery or complete recovery from every failure.

Future controlled tests should include simultaneous SOS requests, a second SOS while an incident is active, missing or invalid coordinates, unauthorized access to another user's incident, incorrect resolution passwords, invalid recipient addresses, SMTP timeouts, worker restarts, and mixed outcomes across multiple contacts. Until these cases are tested and documented, they should remain limitations or future evaluation scenarios rather than verified capabilities.

IX. CONCLUSION AND FUTURE WORK

This paper presented SafeGuardian, a web-based emergency SOS prototype that integrates authenticated access, emergency-contact management, browser geolocation, persistent incident records, and asynchronous email notifications. The application records an SOS incident and creates contact-level notification records, which are processed independently by a notification worker. This architecture makes notification state available for inspection after the initial emergency request has completed.

The available functional validation record reports successful completion of the tested core workflow and no failed notification records in the final controlled validation. These observations support the feasibility of the implementation in the tested environment. They do not establish production-scale reliability, statistically representative success rates, or improved performance under load.

Future work should collect raw per-trial observations, report denominators for every metric, measure request latency and queue-to-send intervals, and evaluate the system under controlled concurrency. Additional testing should examine SMTP failures, worker restarts, retry behavior, duplicate processing,

and privacy safeguards. A controlled comparison with alternative notification architectures could then provide evidence about the benefits and trade-offs of asynchronous processing. Any real-world deployment would also require broader security, operational, and emergency-response evaluation.

ACKNOWLEDGMENT

The authors acknowledge the Department of Computer Science and Engineering, KLH University, for the academic environment and infrastructure used for this project.

REFERENCES

[1] S. Patil, S. Kanawade, Y. Karnawat, K. Deshmukh, S. Chaudhari, and C. Karodpati, "Rakshak: A Multi-Layered Intelligent Framework for Offline-Capable Proactive Emergency Response in Women's Safety Systems," in *Proc. International Conference on Advances in Computing Technology and Artificial Intelligence (COMPUTATIA)*, Atlantis Highlights in Intelligent Systems, 2026, pp. 673–687, doi: 10.2991/978-94-6239-713-2_50.

[2] Y. Saravanan and S. Marjudi, "Women SOS Safety App," *Applied Information Technology and Computer Science*, vol. 5, no. 1, pp. 1240–1254, 2024. [Online]. Available: https://periodical.uthm.edu.my/index.php/aitcs/article/view/12242

[3] "Women Safety System using IoT," in *Proc. International Conference on Applied Artificial Intelligence and Computing (ICAAIC)*, 2022, doi: 10.1109/ICAAIC53929.2022.9792929.

[4] "Enhanced Real-Time Women Safety App: Alert Notification and Emergency Response System With Live Location, Video, and Audio Streaming," in *Proc. International Conference on Data Science and Business Systems (ICDSBS)*, 2025, doi: 10.1109/ICDSBS63635.2025.11031609.

[5] "An SOS Women Safety Device: Automatic Emergency Alerts and Geolocation Sharing with GSM/GPS Integration," in *Proc. International Conference on Intelligent Computing, Networks, and Security (IC-ICNS)*, 2026, doi: 10.1109/IC-ICNS68863.2026.11537940.

[6] P. Jothilakshmi, M. S. Rishikesh, and M. Y. Srikanth, "Cross-Platform Women Safety Mobile Application with Real-Time Alert System and GPS Tracking," in *Proc. International Conference on Emerging Systems and Intelligent Computing (ICESIC)*, 2026, doi: 10.1109/ICESIC67389.2026.11496592.

[7] "Thozha: A Smart Mobile and IoT-Based Safety Ecosystem for Women," in *Proc. 2nd International Conference on Computing and Data Science (ICCDS)*, 2025, doi: 10.1109/ICCDS64403.2025.11209496.

[8] M. S. Farooq, A. Masooma, U. Omer, R. Tehseen, S. A. M. Gilani, and Z. Atal, "The Role of IoT in Woman's Safety: A Systematic Literature Review," *IEEE Access*, vol. 11, pp. 69807–69825, 2023, doi: 10.1109/ACCESS.2023.3252903.

[9] K. Ashwini and S. N. Perla, "An Integrated Mobile Applications for Enhancing Women's Safety – A Comprehensive Approach," in *Proc. IEEE International Conference for Women in Innovation, Technology & Entrepreneurship (ICWITE)*, 2024, pp. 223–228, doi: 10.1109/ICWITE59797.2024.10503411.

[10] G. Uganya, N. Kirubakaran, B. Thomas, and M. Boobalan, "Smart Women Safety Device Using IoT and GPS Tracker," in *Proc. Intelligent Computing and Control for Engineering and Business Systems (ICCEBS)*, 2023, doi: 10.1109/ICCEBS58601.2023.10449302.